Public

Device Identity Provisioning over SPDM

An open, multi-vendor approach to operator-owned device identity

Presenters

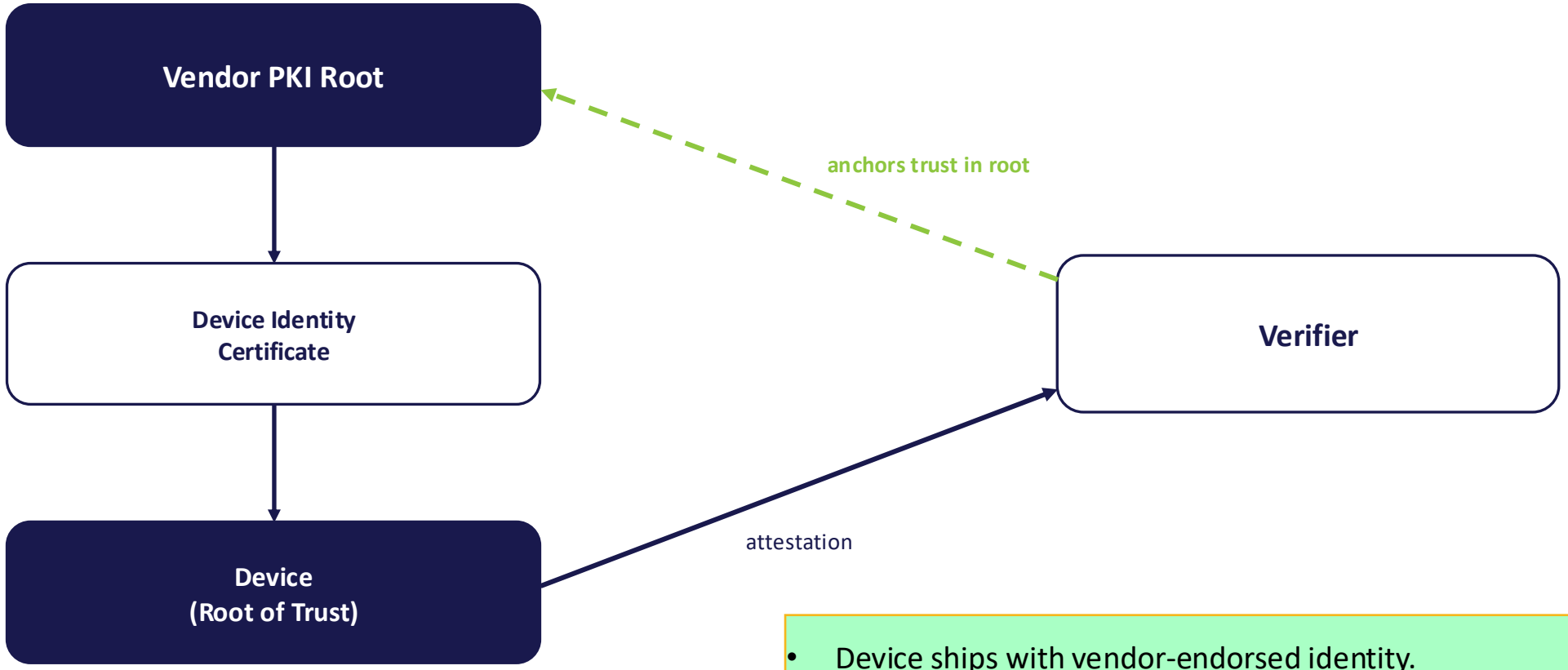
- Fabrizio D Amato (AMD)
- Xiaoyu Ruan (Intel)

Contributors

- Fabrizio D Amato (AMD)
- Jeff Andersen (Google)
- Steven Bellock (NVIDIA)
- Parvathi Bhogaraju (Microsoft)
- Brett Henning (Broadcom)
- Denis Pochuev (Qualcomm)
- Shalini Sharma (Qualcomm)
- Xiaoyu Ruan (Intel)

Public

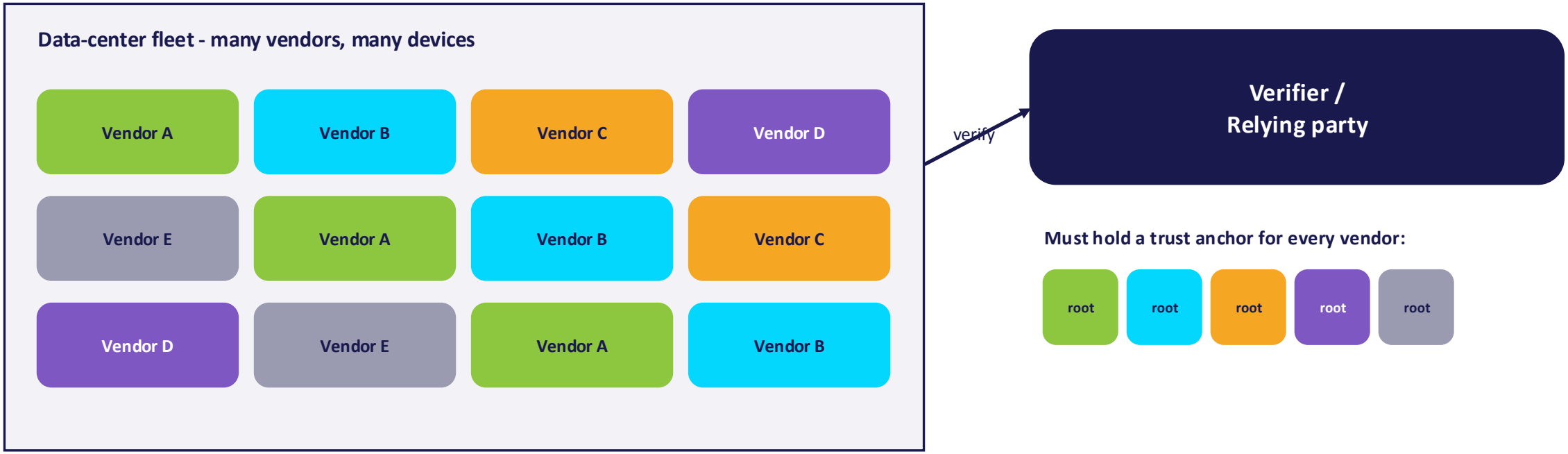
Device identity is vendor-locked



- Device ships with vendor-endorsed identity.
- Verifier chains trust to the vendor PKI root.
- A heterogeneous fleet forces many vendor roots into verifier policy.

Public

The problem at data-center scale

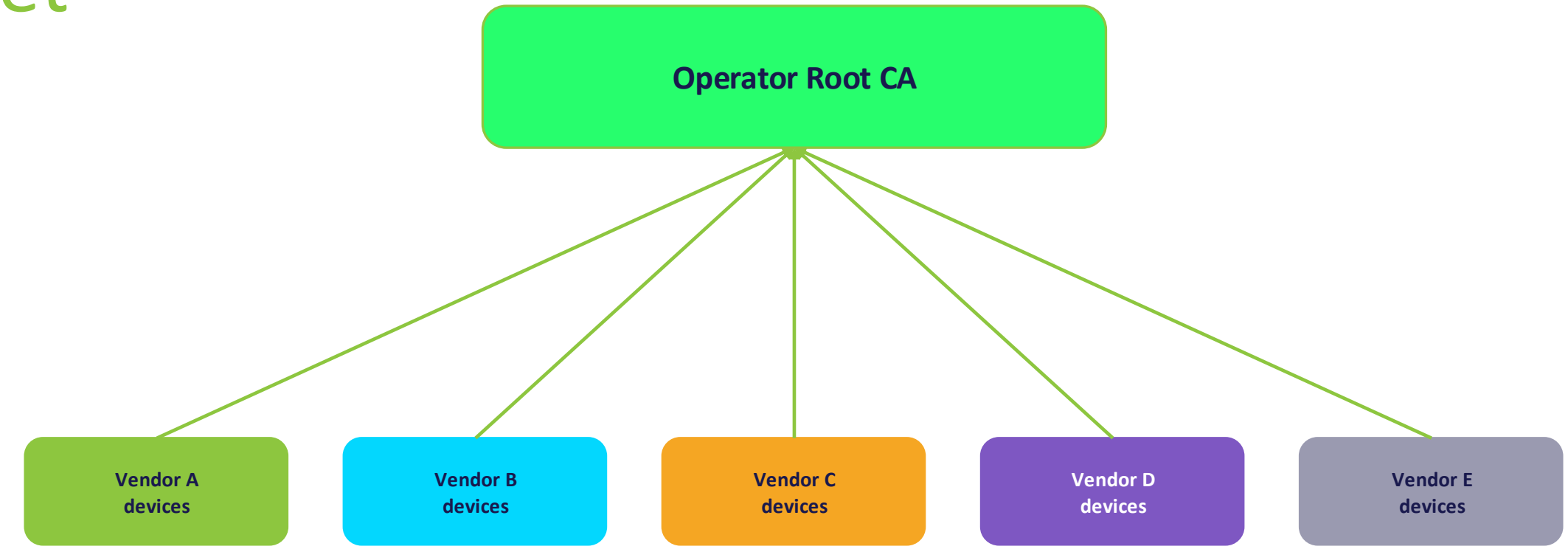


1 Unmanageable - a verifier must hold and maintain a trust anchor for every vendor.

2 Opaque - a relying party can't tell whether two devices belong to the same operator.

Public

Solution: one Operator Root CA for the whole fleet

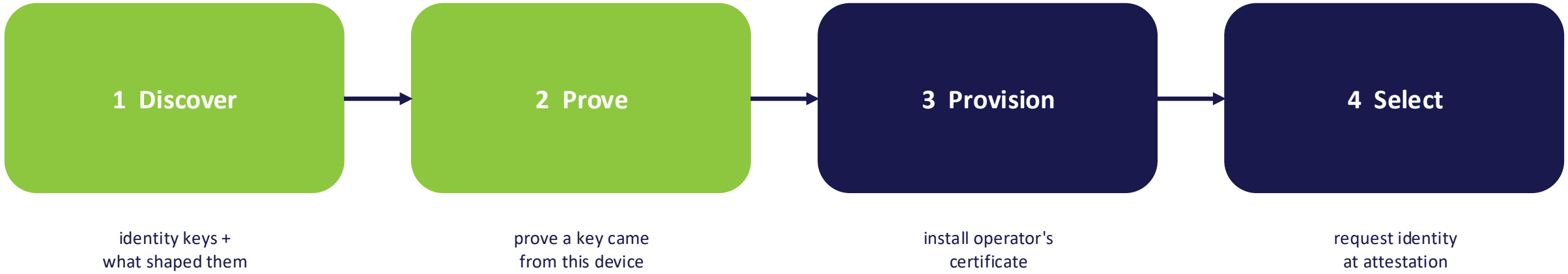


1 **Manageable - the verifier keeps a single operator root, not one per vendor.**

2 **Linkable - devices under one root are provably the same operator's.**

Public

How it works: four steps over SPDM

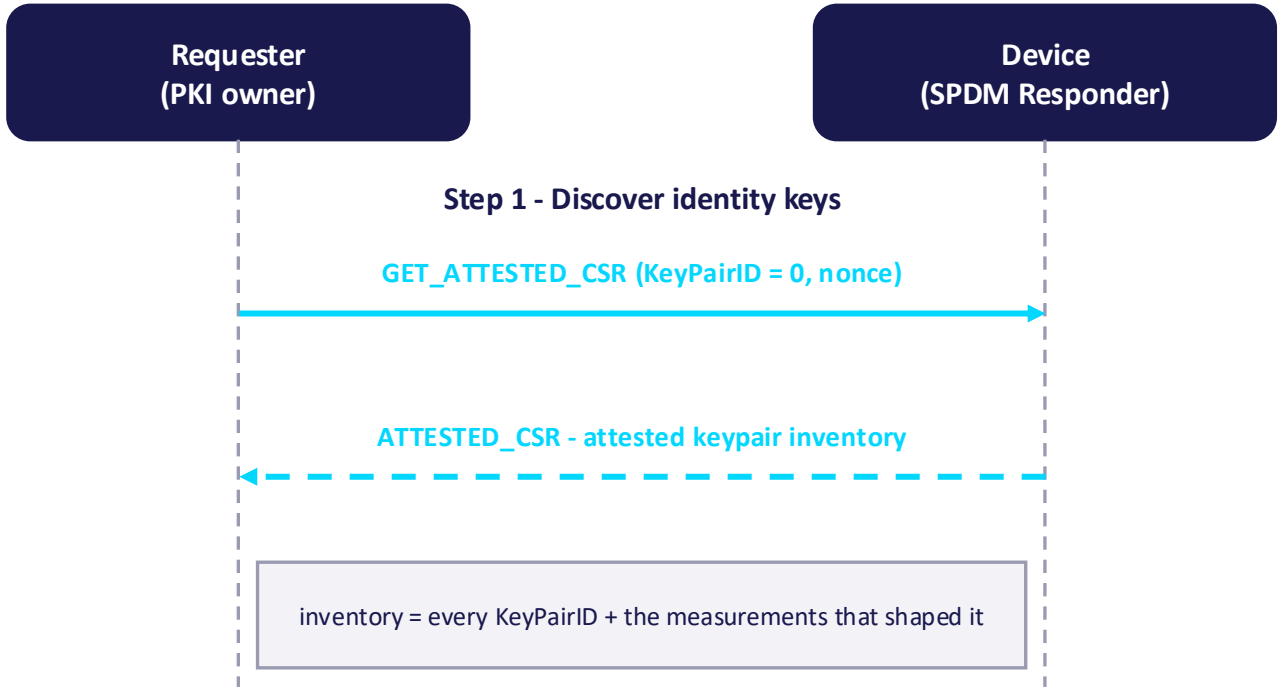


All on top of existing SPDM (>= 1.3)

The specification defines the ATTESTED_CSR flow to bind a selected identity key to device evidence, then uses provisioning and selection flows for PKI-owner certificates

Public

Step 1 - Discovery: enumerate identity keys



Discovery (KeyPairID = 0)

- Device returns every identity keypair
- Each entry carries its derivation attributes
- Requester reads these to choose an anchor

KeyPairID = 0 is the discovery sentinel - ask the device what identities it has.

Public

Key Derivation Attributes: one OID, a bitfield

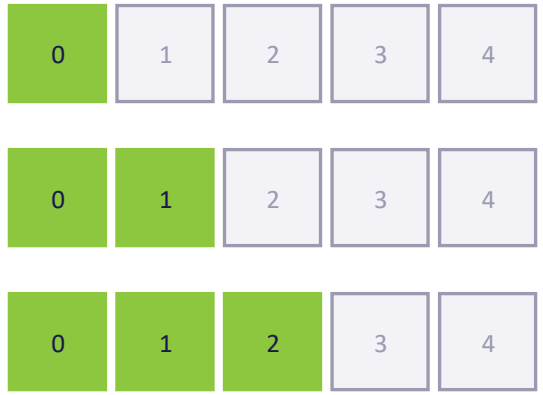
Key Derivation Attributes - one OCP OID (PEN 42623), value is a bitfield

OID 1.3.6.1.4.1.42623. … each bit records one thing that shaped the key

Bit meanings

bit 0	Factory-rooted (UDS)
bit 1	Field entropy mixed in
bit 2	Firmware measurements
bit 3	Security config / fuses
bit 4	Runtime code identity

Example - three device keys



bits set = what shaped the key (cumulative)

Later keys (i.e., a key associated with a higher layer of the cert chain) carry more bits; the Requester matches the bits it needs. The inventory (next) lists each key's attributes.

Public

The attested keypair inventory

EAT (COSE_Sign1) - signed by the device Attestation Key

nonce -> freshness

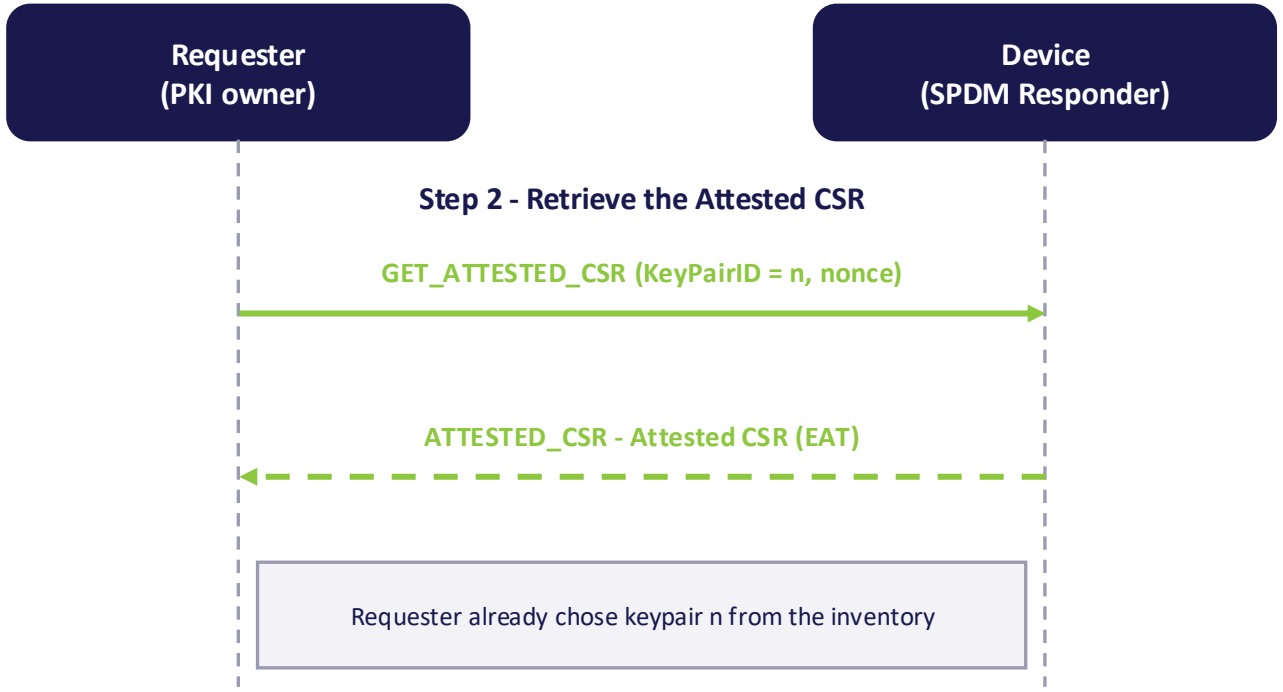
keypair inventory = a list of { KeyPairID, Key Derivation Attributes }

KeyPairID	Key Derivation Attributes (bitfield)				
1	0	1	2	3	4
2	0	1	2	3	4
3	0	1	2	3	4

A generic list - every identity key advertises its Key Derivation Attributes (bit meanings on the previous slide).
At the next step, Requester chooses a keypair with its preferred key derivation attributes and get a CSR.

Public

Step 2 - Retrieve the Attested CSR



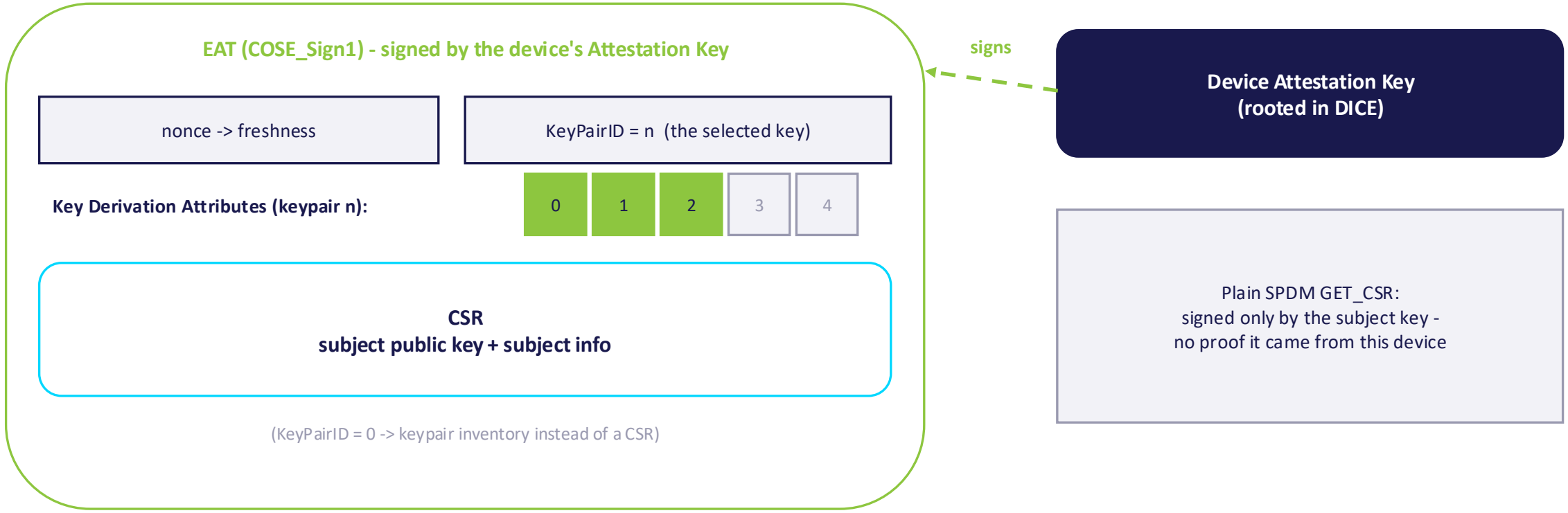
Retrieve (KeyPairID = n)

- Requester picks the keypair that fits its anchor
- Device returns a CSR for that keypair
- The CSR is wrapped in a nonce-bound EAT
- Signed by the device Attestation Key

The same command with a real KeyPairID returns an attested CSR for that key.

Public

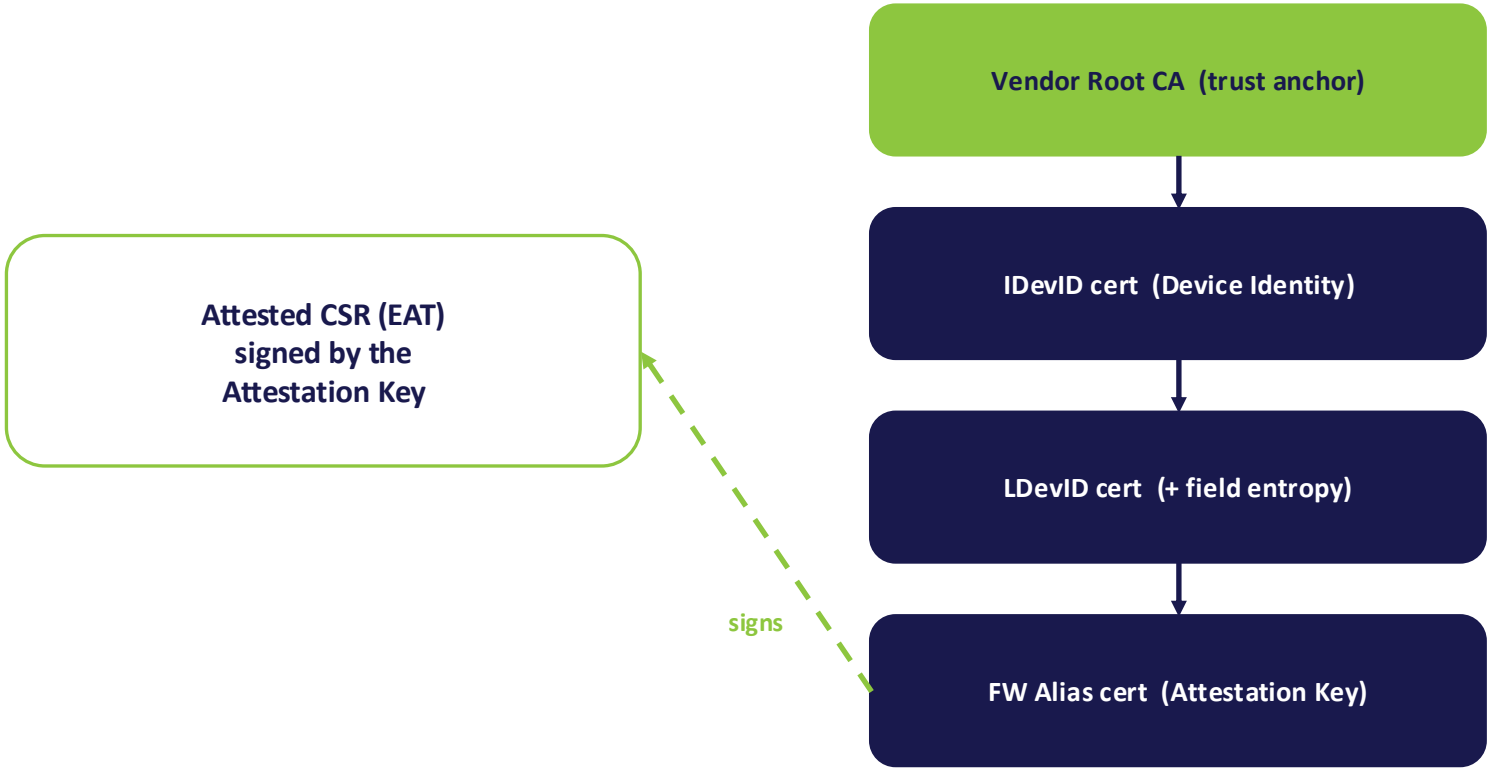
Inside the response: the Attested CSR



The device attests the CSR in a nonce-bound EAT - proof it came from THIS device.

Public

Establishing trust: the key, then the chain



Verifier establishes trust in order:

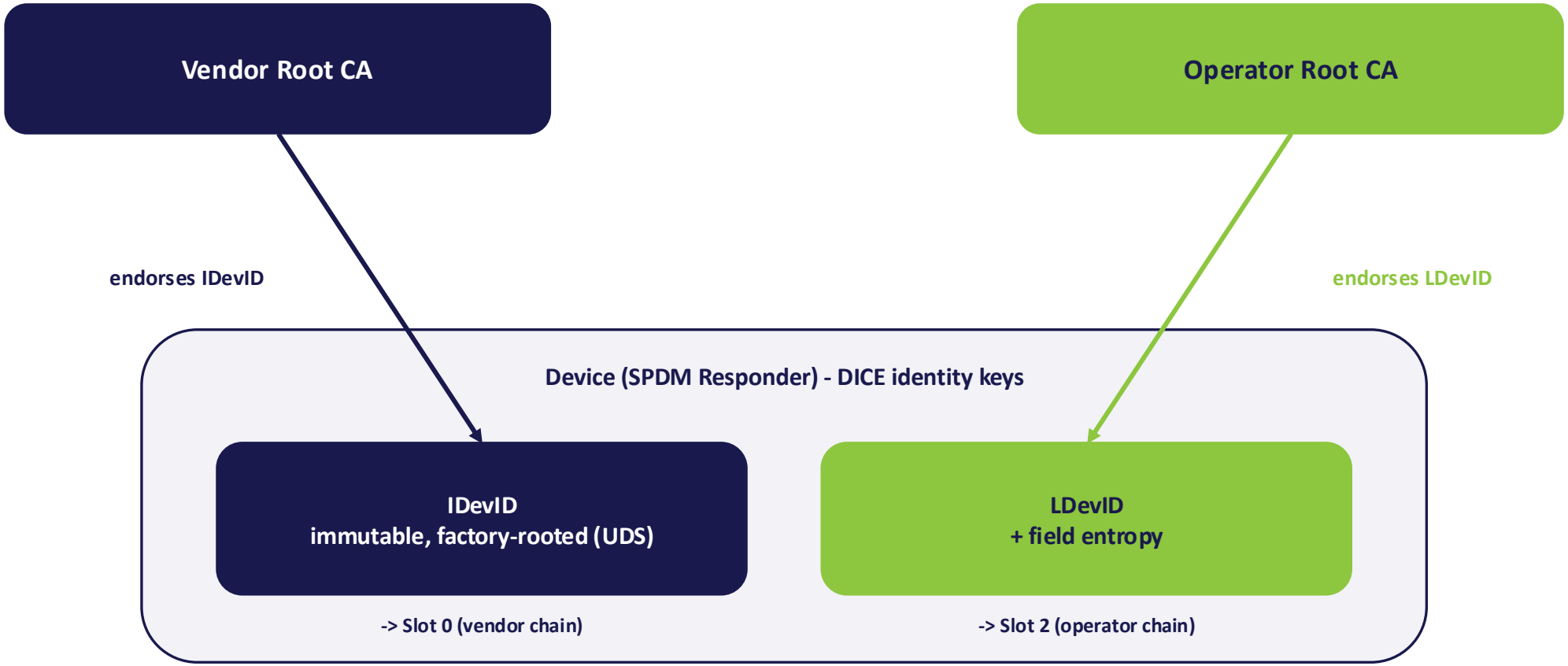
1 Verify the EAT signature with the Attestation Key -> trust the KEY

2 Validate the Alias cert chain up to the Root CA -> trust the CHAIN

First trust the key that signed the response, then the chain that vouches for the key.

Public

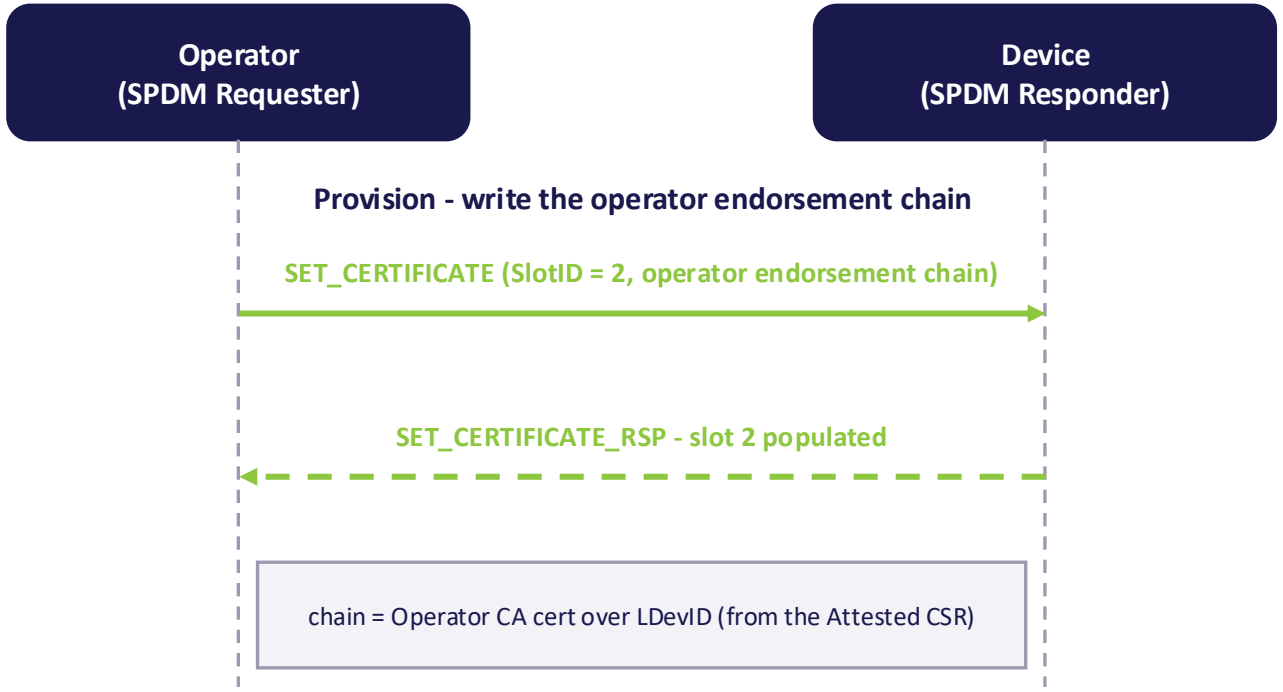
Two endorsements: Vendor CA and Operator PKI



One device, two endorsements: the Vendor vouches for IDevID, the Operator becomes a co-root for the LDevID it owns.

Public

Step 3: Provision: SET_CERTIFICATE to Slot 2



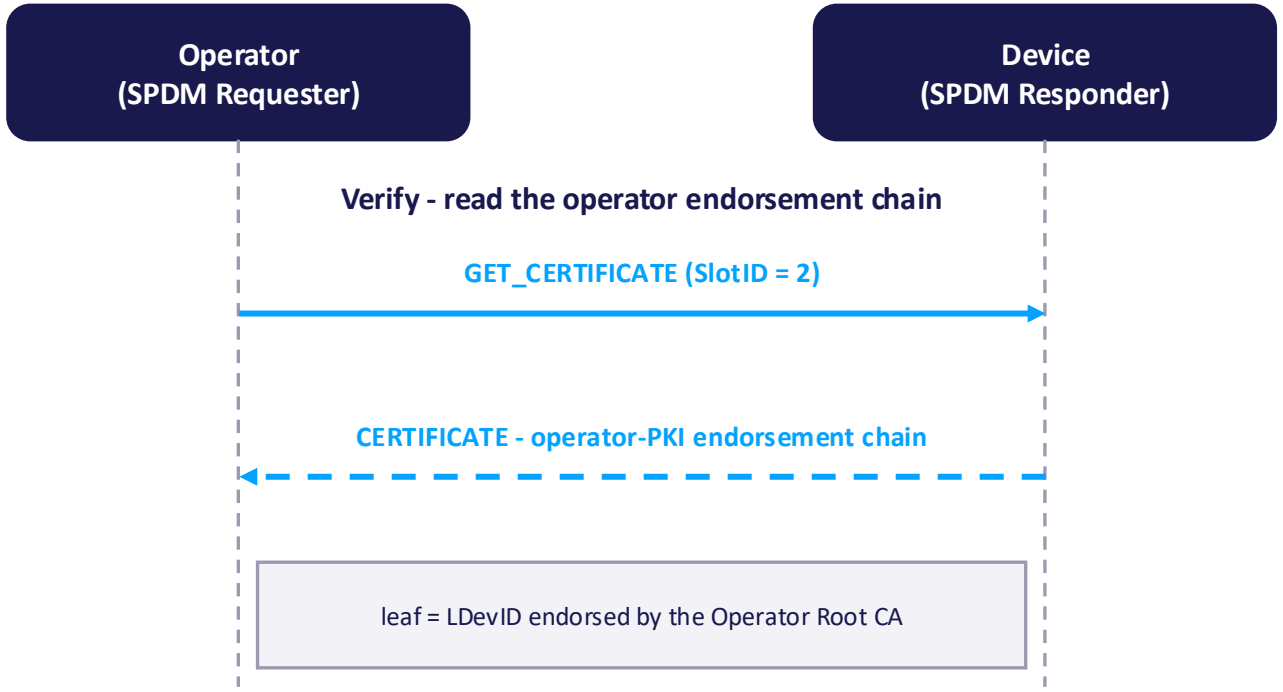
SET_CERTIFICATE (Slot 2)

- Operator CA issued a cert over LDevID
- Operator writes that chain into slot 2
- Certs from after LDevID to leaf are same as slot 0
- Slot 0 (vendor IDevID) is left untouched
- Device now holds both roots at once

The operator provisions its own endorsement into slot 2 - the vendor chain in slot 0 stays intact.

Public

Step 4: Select: GET_CERTIFICATE from Slot 2



GET_CERTIFICATE (Slot 2)

- Operator verifier reads slot 2
- Device returns the operator-rooted chain
- Root = Operator Root CA
- Certs after LDevID do not change (same as slot 0)
- Whole fleet verifies against one root

Slot 2 returns the operator-rooted chain - the fleet verifies against a single operator root, no vendor PKI needed.

Public

Why it matters

- Consistent verification: one PKI root across a multi-vendor fleet
- Compromise insulation: vendor-PKI trust becomes point-in-time, not perpetual
- Supply-chain insulation: anchor on a later key (e.g. LDevID + field entropy)
- Implicit attestation: choosing the anchor blesses the measurements that shaped that key

Public

Flexible trust: many owners, one device

- A device can serve multiple PKI owners at once - e.g. operator and tenant
- It caches owner's certificate and serves the right one per attestation request
- Operator cert chain is always provisioned to slot 2.
- New operator can delete previous owner’s chain and re-provision slot 2.
- Tenant may provision tenant-rooted cert chain to any available slot.
- Certificate deactivation: if the measurements change, the key changes and the cert deactivates
- Deactivation is reversible - not the same as revocation or expiry

Public

Confidential Computing: proof you're on the right device

- A tenant runs a confidential workload on a machine rented from an operator
- As a PKI owner, the tenant gets cryptographic proof of custody - bound to this operator-owned device
- Forgery-resistant: a field-entropy-derived anchor survives extraction of the factory secret (UDS)
- Tamper-evident: the tenant's certificate deactivates if the measured configuration changes

Public

Status and Next Steps

- OCP Device Identity Provisioning spec 80% complete (?)
- Depends on SPDM command GET_ATTESTATION
  - Either OCP-defined VDM of SPDM or a new native SPDM command
- OCP has requested DMTF to support in SPDM 1.5:
  - Define GET_ATTESTATION as an SPDM command.
  - Allow GET_CERT and SET_CERT to present partial cert chain, for saving space and improving performance. (A PQC cert can be as large as 8KB.)

Public

Call to action: get involved

- Built on open standards: SPDM 1.3 (DMTF), DICE (TCG), EAT/CWT (IETF), Caliptra reference implementation
- Where: OCP Security project - spec draft
- Who: AMD, NVIDIA, MSFT, Google, Broadcom, Qualcomm, Intel, MediaTek - open to more
- What's next: provisioning, attestation-time selection, confidential-compute details
- Read the spec, give feedback, implement it, adopt it

Public

Thank You!

